

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

**Дисциплина:
«Программирование»**

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Работа с файлами и структуры данных на основе указателей»

Вариант: 2-10-8-4

Выполнил:

студент гр. N3151

Агакарян Арам Арманович



(подпись)

Проверил:

инженер ФБИТ

Дронов Вадим Юрьевич

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

ВВЕДЕНИЕ

Цель работы – Разработать на языке C для ОС Linux программу, которая читает из файла заданного формата состояние заданной структуры данных, выполняет манипуляции со структурой в памяти, считывая команды из стандартного потока ввода, и сохраняет результат в файл. Также для проверки программы написать скрипт на языке Python, который позволяет создать файл заданного формата со случайными данными.

Задание

Вариант - 2-10-8-4

Двусвязный список, Автомобильный номер РФ

Команда: `reorder_asc`

Описание: Расположить элементы списка по возрастанию.

Содержимое файла:

Поле `cnt` (целое без знака [4 байта]). Количество пар полей индексов и смещений строк, расположенных сразу после этого поля.

Индексы `i0, ..., iN-1` (целые без знака [2 байта]). Индексы соответствующих строк `s0, ..., sN-1`, определяющие порядок строк в списке.

Смещения `o0, ..., oN-1` (целые без знака [4 байта]). Смещения соответствующих строк в файле. Каждый элемент массива `oi` содержит смещение в байтах от начала файла строки `si`.

Строки `s0, ..., sN-1`. Строки в кодировке UTF-8 с нулевым символом в конце располагаются в файле с возможными промежутками между ними.

1 MAKEFILE

Make-файл... Можно привести как рисунок или листинг (лучше листинг).

```
CC = gcc
```

```
CFLAGS = -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE
```

```
TARGET = prg4aaaN3151
```

```
all: $(TARGET)
```

```
$(TARGET): prg4aaaN3151.c
```

```
$(CC) $(CFLAGS) prg4aaaN3151.c -o $(TARGET)
```

```
clean:
```

```
rm -f $(TARGET) out.txt cmds.txt *.bin
```

```
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via 🐞 v3.12.3 took 5s
```

```
★ 14:57:40 $ cat Makefile
```

```
CC = gcc
```

```
CFLAGS = -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE
```

```
TARGET = prg4aaaN3151
```

```
all: $(TARGET)
```

```
$(TARGET): prg4aaaN3151.c
```

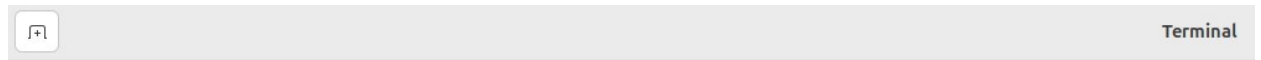
```
$(CC) $(CFLAGS) prg4aaaN3151.c -o $(TARGET)
```

```
clean:
```

```
rm -f $(TARGET) out.txt cmds.txt *.bin
```

2 ПРИМЕРЫ РАБОТЫ ПРОГРАММ

Примеры работы программ на различных исходных данных (скриншоты). Ссылаться на них как на рисунки.



```
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:41:32 $ make clean
rm -f prg4aaaN3151 out.txt cmds.txt *.bin

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:41:34 $ make
gcc -Wall -Wextra -Werror -std=c11 -D_GNU_SOURCE prg4aaaN3151.c -o prg4aaaN3151

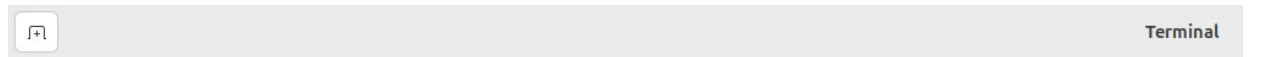
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:41:59 $ ./prg4aaaN3151 -v
Агакарян Арам Арманович, гр. N3151
Вариант: 2-10-8-4

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:42:24 $ python3 gen.py -v
Агакарян Арам Арманович, гр. N3151
Вариант: 2-10-8-4

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:42:30 $ ./prg4aaaN3151
Ошибка: не задано имя файла.

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:42:42 $ ./prg4aaaN3151 a b
Ошибка: лишние аргументы.

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:42:48 $ python3 gen.py
Ошибка: нужно указать ровно одно имя файла.
```



```
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:42:56 $ python3 gen.py -n 5 test.bin
Файл: test.bin
cnt = 5
Заголовок (i,o)-пар: с 0x4 по 0x22 (30 байт)
data_first: размер 11 байт, смещение 0x22
data_last: размер 23 байт, смещение 0x69

Заголовок (k – позиция в файле, idx – позиция в списке, off – смещение строки):
k= 0 idx= 1 off=0x0000002d ( 45) s_k=н579кн55RUS
k= 1 idx= 4 off=0x0000003c ( 60) s_k=с320св65RUS
k= 2 idx= 2 off=0x0000004b ( 75) s_k=с631ху21RUS
k= 3 idx= 3 off=0x0000005a ( 90) s_k=х911мн12RUS
k= 4 idx= 0 off=0x00000080 (128) s_k=м538ео78RUS

Порядок строк в списке после загрузки:
list[ 0] = м538ео78RUS
list[ 1] = н579кн55RUS
list[ 2] = с631ху21RUS
list[ 3] = х911мн12RUS
list[ 4] = с320св65RUS

Успешно: файл 'test.bin' создан, строк: 5
```

```
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:50:25 $ ./prg4aaaN3151 test.bin
> dump
0x59b2efeda640 (nil) 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c631xy21RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 x911MH12RUS
0x59b2efeda740 0x59b2efeda700 (nil) c320cb65RUS
> push_back a123bc78RUS
> dump
0x59b2efeda640 (nil) 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c631xy21RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 x911MH12RUS
0x59b2efeda740 0x59b2efeda700 0x59b2efeda490 c320cb65RUS
0x59b2efeda490 0x59b2efeda740 (nil) a123bc78RUS
> push_front t111tt77RUS
> dump
0x59b2efeda5a0 (nil) 0x59b2efeda640 t111tt77RUS
0x59b2efeda640 0x59b2efeda5a0 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c631xy21RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 x911MH12RUS
0x59b2efeda740 0x59b2efeda700 0x59b2efeda490 c320cb65RUS
0x59b2efeda490 0x59b2efeda740 (nil) a123bc78RUS
> pop_back
> dump
0x59b2efeda5a0 (nil) 0x59b2efeda640 t111tt77RUS
0x59b2efeda640 0x59b2efeda5a0 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c631xy21RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 x911MH12RUS
0x59b2efeda740 0x59b2efeda700 (nil) c320cb65RUS
> pop_front
> dump
0x59b2efeda640 (nil) 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c631xy21RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 x911MH12RUS
0x59b2efeda740 0x59b2efeda700 (nil) c320cb65RUS
> reorder_asc
> dump
0x59b2efeda640 (nil) 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 h579KH55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c320cb65RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 c631xy21RUS
0x59b2efeda740 0x59b2efeda700 (nil) x911MH12RUS
> dump out.txt
> exit
```

Terminal

```
~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 1m54s
14:52:36 $ cat out.txt
0x59b2efeda640 (nil) 0x59b2efeda680 m538eo78RUS
0x59b2efeda680 0x59b2efeda640 0x59b2efeda6c0 н579кн55RUS
0x59b2efeda6c0 0x59b2efeda680 0x59b2efeda700 c320св65RUS
0x59b2efeda700 0x59b2efeda6c0 0x59b2efeda740 c631xy21RUS
0x59b2efeda740 0x59b2efeda700 (nil) x911мн12RUS

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:52:46 $ ./prg4aaaN3151 test.bin
> dump
0x599441f43640 (nil) 0x599441f43680 m538eo78RUS
0x599441f43680 0x599441f43640 0x599441f436c0 н579кн55RUS
0x599441f436c0 0x599441f43680 0x599441f43700 c320св65RUS
0x599441f43700 0x599441f436c0 0x599441f43740 c631xy21RUS
0x599441f43740 0x599441f43700 (nil) x911мн12RUS
> push_back 123
Ошибка: неверный формат номера: 123.
> pop_back abc
Ошибка: лишние аргументы.
> pop_front abc
Ошибка: лишние аргументы.
> dump a b
Ошибка: лишние аргументы.
> exit

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 38s
14:53:59 $ : > empty.bin

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:54:18 $ ./prg4aaaN3151 empty.bin
> dump
> exit

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 5s
14:54:34 $ ./prg4aaaN3151 notfile.bin
> dump
> exit
```



```
14:54:34 $ ./prg4aaaN3151 notfile.bin
> dump
> exit

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 4s
14:54:49 $ ./prg4aaaN3151 test.bin
> dump
0x57df1a139640 (nil) 0x57df1a139680 m538eo78RUS
0x57df1a139680 0x57df1a139640 0x57df1a1396c0 h579KH55RUS
0x57df1a1396c0 0x57df1a139680 0x57df1a139700 c320cb65RUS
0x57df1a139700 0x57df1a1396c0 0x57df1a139740 c631xy21RUS
0x57df1a139740 0x57df1a139700 (nil) x911MH12RUS
> pop_back
> pop_back
> pop_back
> pop_back
> pop_back
> dump
> exit

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 22s
14:55:27 $ ls -l test.bin
-rw-rw-r-- 1 harry harry 0 May 31 14:55 test.bin

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
14:55:32 $ cat > cmds.txt5
push_back a111aa11RUS
push_back a111aa11RUS
reorder_asc
exit
^Z
[1]+  Stopped                  cat > cmds.txt5

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 45s
+ 14:56:46 $ ./prg4aaaN3151 test5.bin <cmds.txt5
> > >

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3
+ 14:57:27 $ ./prg4aaaN3151 test5.bin
> dump
0x587550bd8590 (nil) 0x587550bd85d0 a111aa11RUS
0x587550bd85d0 0x587550bd8590 (nil) a111aa11RUS
> exit

~/Desktop/prg4aaaN3151final via C v13.3.0-gcc via v3.12.3 took 5s
+ 14:57:40 $
```

3 ИСХОДНЫЙ КОД ПРОГРАММЫ

Код (с комментариями!). В данном случае разрешается вставить большой код в виде листинга сюда.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <unistd.h>
#include <errno.h>
#include <ctype.h>
#include <getopt.h>

typedef struct Node
{
    char *data;
    struct Node *next;
    struct Node *prev;
} Node;

typedef struct
{
    Node *head;
    Node *tail;
    int modified;
    unsigned char *data_first;
    size_t data_first_len;
    unsigned char *data_last;
    size_t data_last_len;
} DoubleList;

// Допустимые буквы
static const char *const ALLOWED_LETTERS[] = {
    "\xD0\xB0", // а
```

```

"\xD0\xB2", // в
"\xD0\xB5", // е
"\xD0\xBA", // к
"\xD0\xBC", // м
"\xD0\xBD", // н
"\xD0\xBE", // о
"\xD1\x80", // р
"\xD1\x81", // с
"\xD1\x82", // т
"\xD1\x83", // у
"\xD1\x85", // х
};

static const size_t ALLOWED_LETTERS_COUNT = sizeof(ALLOWED_LETTERS) /
sizeof(ALLOWED_LETTERS[0]);

static int is_allowed_letter(const char *p)
{
    for (size_t i = 0; i < ALLOWED_LETTERS_COUNT; i++)
    {
        if (memcmp(p, ALLOWED_LETTERS[i], 2) == 0)
            return 1;
    }
    return 0;
}

// очистка памяти
void list_clear(DoubleList *list)
{
    Node *curr = list->head;
    while (curr)
    {
        Node *next = curr->next;
        free(curr->data);
        free(curr);
        curr = next;
    }
}

```

```

    }
    list->head = list->tail = NULL;
    free(list->data_first);
    list->data_first = NULL;
    list->data_first_len = 0;
    free(list->data_last);
    list->data_last = NULL;
    list->data_last_len = 0;
}

int is_valid_car_number(const char *s)
{
    if (!s || strlen(s) != 14)
        return 0;
    if (!is_allowed_letter(s))
        return 0;
    if (!isdigit((unsigned char)s[2]) || !isdigit((unsigned char)s[3]) || !isdigit((unsigned char)s[4]))
        return 0;
    if (!is_allowed_letter(s + 5))
        return 0;
    if (!is_allowed_letter(s + 7))
        return 0;
    if (!isdigit((unsigned char)s[9]) || !isdigit((unsigned char)s[10]))
        return 0;
    return (strcmp(s + 11, "RUS") == 0);
}

// списки
void push_back(DoubleList *list, const char *str)
{
    if (!is_valid_car_number(str))
    {
        fprintf(stderr, "Ошибка: неверный формат номера: %s.\n", str);
        return;
    }
}

```

```

    }
    Node *n = malloc(sizeof(Node));
    if (!n)
    {
        fprintf(stderr, "Ошибка: недостаточно памяти.\n");
        exit(1);
    }
    n->data = strdup(str);
    if (!n->data)
    {
        free(n);
        exit(1);
    }
    n->next = NULL;
    n->prev = list->tail;
    if (list->tail)
        list->tail->next = n;
    else
        list->head = n;
    list->tail = n;
    list->modified = 1;
}

void push_front(DoubleList *list, const char *str)
{
    if (!is_valid_car_number(str))
    {
        fprintf(stderr, "Ошибка: неверный формат номера: %s.\n", str);
        return;
    }
    Node *n = malloc(sizeof(Node));
    if (!n)
    {
        fprintf(stderr, "Ошибка: недостаточно памяти.\n");

```

```

        exit(1);
    }
    n->data = strdup(str);
    if (!n->data)
    {
        free(n);
        exit(1);
    }
    n->prev = NULL;
    n->next = list->head;
    if (list->head)
        list->head->prev = n;
    else
        list->tail = n;
    list->head = n;
    list->modified = 1;
}

```

```

void list_reorder_asc(DoubleList *list)
{
    if (!list->head)
        return;
    int sw;
    Node *p;
    do
    {
        sw = 0;
        p = list->head;
        while (p->next)
        {
            if (strcmp(p->data, p->next->data) > 0)
            {
                char *t = p->data;
                p->data = p->next->data;

```

```

        p->next->data = t;
        sw = 1;
        list->modified = 1;
    }
    p = p->next;
}
} while (sw);
}

```

```

static void die_corrupt(FILE *f, uint16_t *idxs, uint32_t *offs, char **strs,
                        uint32_t cnt, uint32_t *order, char *seen)
{
    fprintf(stderr, "Ошибка: некорректный формат файла.\n");
    if (strs)
        for (uint32_t i = 0; i < cnt; i++)
            free(strs[i]);
    free(idxs);
    free(offs);
    free(strs);
    free(order);
    free(seen);
    if (f)
        fclose(f);
    exit(1);
}

```

```

void load_from_file(const char *fn, DoubleList *list)
{
    FILE *f = fopen(fn, "rb");
    if (!f)
        return;

    uint32_t cnt;
    if (fread(&cnt, 4, 1, f) != 1)

```

```

{
    fclose(f);
    return;
}
if (cnt == 0)
{
    fclose(f);
    return;
}

uint16_t *idxs = malloc(cnt * sizeof(uint16_t));
uint32_t *offs = malloc(cnt * sizeof(uint32_t));
char **strs = calloc(cnt, sizeof(char *));
uint32_t *order = malloc(cnt * sizeof(uint32_t));
char *seen = calloc(cnt, 1);

if (!idxs || !offs || !strs || !order || !seen)
{
    fprintf(stderr, "Ошибка: недостаточно памяти.\n");
    free(idxs);
    free(offs);
    free(strs);
    free(order);
    free(seen);
    fclose(f);
    exit(1);
}

for (uint32_t k = 0; k < cnt; k++)
{
    if (fread(&idxs[k], 2, 1, f) != 1 || fread(&offs[k], 4, 1, f) != 1)
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
}

```

```

for (uint32_t k = 0; k < cnt; k++)
{
    if (idxs[k] >= cnt || seen[idxs[k]])
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
    seen[idxs[k]] = 1;
    order[k] = k;
}

for (uint32_t i = 0; i + 1 < cnt; i++)
{
    for (uint32_t j = 0; j + 1 < cnt - i; j++)
    {
        if (offs[order[j]] > offs[order[j + 1]])
        {
            uint32_t t = order[j];
            order[j] = order[j + 1];
            order[j + 1] = t;
        }
    }
}

uint32_t header_end = 4 + cnt * 6;
if (offs[order[0]] < header_end)
    die_corrupt(f, idxs, offs, strs, cnt, order, seen);

list->data_first_len = offs[order[0]] - header_end;
if (list->data_first_len > 0)
{
    list->data_first = malloc(list->data_first_len);
    if (!list->data_first)
    {
        fprintf(stderr, "Ошибка: недостаточно памяти.\n");
        exit(1);
    }
}

```

```

    if (fseek(f, header_end, SEEK_SET) != 0 ||
        fread(list->data_first, 1, list->data_first_len, f) != list->data_first_len)
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
}

for (uint32_t k = 0; k < cnt; k++)
{
    if (fseek(f, offs[k], SEEK_SET) != 0)
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);

    size_t cp = 16, ln = 0;
    char *b = malloc(cp);
    if (!b)
    {
        fprintf(stderr, "Ошибка: недостаточно памяти.\n");
        exit(1);
    }
    int c;
    while ((c = fgetc(f)) != EOF && c != '\0')
    {
        if (ln + 1 >= cp)
        {
            char *tmp = realloc(b, cp *= 2);
            if (!tmp)
            {
                free(b);
                fprintf(stderr, "Ошибка: недостаточно памяти.\n");
                exit(1);
            }
            b = tmp;
        }
        b[ln++] = c;
    }
    if (c != '\0')

```

```

    {
        free(b);
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
    }
    b[ln] = '\0';
    if (!is_valid_car_number(b))
    {
        free(b);
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
    }
    strs[k] = b;
}

if (cnt >= 2)
{
    uint32_t penult_k = order[cnt - 2];
    uint32_t last_k = order[cnt - 1];
    uint32_t penult_end = offs[penult_k] + (uint32_t)strlen(strs[penult_k]) + 1;
    if (offs[last_k] < penult_end)
        die_corrupt(f, idxs, offs, strs, cnt, order, seen);
    list->data_last_len = offs[last_k] - penult_end;
    if (list->data_last_len > 0)
    {
        list->data_last = malloc(list->data_last_len);
        if (!list->data_last)
        {
            fprintf(stderr, "Ошибка: недостаточно памяти.\n");
            exit(1);
        }
        if (fseek(f, penult_end, SEEK_SET) != 0 ||
            fread(list->data_last, 1, list->data_last_len, f) != list->data_last_len)
            die_corrupt(f, idxs, offs, strs, cnt, order, seen);
    }
}
}

```

```

uint32_t *inv = malloc(cnt * sizeof(uint32_t));
if (!inv)
{
    fprintf(stderr, "Ошибка: недостаточно памяти.\n");
    exit(1);
}
for (uint32_t k = 0; k < cnt; k++)
    inv[idxs[k]] = k;
for (uint32_t p = 0; p < cnt; p++)
    push_back(list, strs[inv[p]]);
free(inv);

for (uint32_t k = 0; k < cnt; k++)
    free(strs[k]);
free(idxs);
free(offsets);
free(strs);
free(order);
free(seen);
fclose(f);
list->modified = 0;
}

void save_to_file(const char *fn, DoubleList *list)
{
    FILE *f = fopen(fn, "wb");
    if (!f)
    {
        fprintf(stderr, "Ошибка: не удалось создать файл.\n");
        return;
    }

    uint32_t cnt = 0;
    for (Node *c = list->head; c; c = c->next)

```

```

    cnt++;

if (cnt == 0)
{
    fclose(f);
    return;
}

uint32_t *offs = malloc(cnt * sizeof(uint32_t));
if (!offs)
{
    fclose(f);
    return;
}

uint32_t header_end = 4 + cnt * 6;
uint32_t pos = header_end + (uint32_t)list->data_first_len;
if (cnt == 1)
    pos += (uint32_t)list->data_last_len;
Node *c = list->head;
for (uint32_t k = 0; k < cnt; k++)
{
    if (k == cnt - 1 && cnt >= 2)
        pos += (uint32_t)list->data_last_len;
    offs[k] = pos;
    pos += (uint32_t)strlen(c->data) + 1;
    c = c->next;
}

if (fwrite(&cnt, 4, 1, f) != 1)
    goto out;
for (uint16_t k = 0; k < cnt; k++)
{
    uint16_t idx = k;

```

```

    if (fwrite(&idx, 2, 1, f) != 1)
        goto out;
    if (fwrite(&offs[k], 4, 1, f) != 1)
        goto out;
}
if (list->data_first_len > 0 &&
    fwrite(list->data_first, 1, list->data_first_len, f) != list->data_first_len)
    goto out;
// Для cnt == 1 data_last идёт сразу за data_first перед единственной строкой
if (cnt == 1 && list->data_last_len > 0 &&
    fwrite(list->data_last, 1, list->data_last_len, f) != list->data_last_len)
    goto out;

c = list->head;
for (uint32_t k = 0; k < cnt; k++)
{
    if (k == cnt - 1 && cnt >= 2 && list->data_last_len > 0)
    {
        if (fwrite(list->data_last, 1, list->data_last_len, f) != list->data_last_len)
            goto out;
    }
    size_t slen = strlen(c->data) + 1;
    if (fwrite(c->data, 1, slen, f) != slen)
        goto out;
    c = c->next;
}

out:
    free(offs);
    fclose(f);
}

int main(int argc, char *argv[])
{

```

```

int opt;
while ((opt = getopt(argc, argv, "v")) != -1)
{
    switch (opt)
    {
        case 'v':
            printf("Агакарян Арам Арманович, гр. N3151\nВариант: 2-10-8-4\n");
            return 0;
        default:
            return 1;
    }
}

```

```

if (optind >= argc)
{
    fprintf(stderr, "Ошибка: не задано имя файла.\n");
    return 1;
}
if (optind + 1 != argc)
{
    fprintf(stderr, "Ошибка: лишние аргументы.\n");
    return 1;
}

```

```

DoubleList list = {NULL, NULL, 0, NULL, 0, NULL, 0};
load_from_file(argv[optind], &list);

```

```

char *line = NULL;
size_t len = 0;
while (printf("> "), getline(&line, &len, stdin) != -1)
{
    line[strcspn(line, "\r\n")] = 0;
    char *cmd = strtok(line, " "), *p;
    if (!cmd)

```

```

    continue;

if (strcmp(cmd, "push_back") == 0)
{
    while ((p = strtok(NULL, " "))
        push_back(&list, p);
}
else if (strcmp(cmd, "push_front") == 0)
{
    char **tk = NULL;
    int tc = 0;
    while ((p = strtok(NULL, " "))
    {
        char **tmp = realloc(tk, sizeof(char *) * (tc + 1));
        if (!tmp)
        {
            free(tk);
            exit(1);
        }
        tk = tmp;
        tk[tc++] = p;
    }
    for (int i = tc - 1; i >= 0; i--)
        push_front(&list, tk[i]);
    free(tk);
}
else if (strcmp(cmd, "pop_back") == 0)
{
    if (strtok(NULL, " "))
        fprintf(stderr, "Ошибка: лишние аргументы.\n");
    else if (list.tail)
    {
        Node *t = list.tail;
        list.tail = t->prev;
    }
}

```

```

        if (list.tail)
            list.tail->next = NULL;
        else
            list.head = NULL;
        free(t->data);
        free(t);
        list.modified = 1;
    }
}
else if (strcmp(cmd, "pop_front") == 0)
{
    if (strtok(NULL, " "))
        fprintf(stderr, "Ошибка: лишние аргументы.\n");
    else if (list.head)
    {
        Node *t = list.head;
        list.head = t->next;
        if (list.head)
            list.head->prev = NULL;
        else
            list.tail = NULL;
        free(t->data);
        free(t);
        list.modified = 1;
    }
}
else if (strcmp(cmd, "reorder_asc") == 0)
{
    if (strtok(NULL, " "))
        fprintf(stderr, "Ошибка: лишние аргументы.\n");
    else
        list_reorder_asc(&list);
}
else if (strcmp(cmd, "dump") == 0)

```

```

{
    p = strtok(NULL, " ");
    if (p)
    {
        if (strtok(NULL, " "))
        {
            fprintf(stderr, "Ошибка: лишние аргументы.\n");
            continue;
        }
        FILE *df = fopen(p, "w");
        if (df)
        {
            for (Node *c = list.head; c; c = c->next)
                fprintf(df, "%p %p %p %s\n", (void *)c, (void *)c->prev, (void *)c->next, c->data);
            fclose(df);
        }
        else
            perror("fopen");
    }
    else
    {
        for (Node *c = list.head; c; c = c->next)
            printf("%p %p %p %s\n", (void *)c, (void *)c->prev, (void *)c->next, c->data);
    }
}
else if (strcmp(cmd, "exit") == 0)
    break;
else
    fprintf(stderr, "Ошибка: неподдерживаемая команда: '%s'.\n", cmd);
}

if (list.modified)
    save_to_file(argv[optind], &list);
list_clear(&list);

```

```
    free(line);  
    return 0;  
}
```

py

```
#!/usr/bin/python3  
import sys  
import random  
import struct  
import getopt  
  
STUDENT_NAME = "Агакарян Арам Арманович"  
STUDENT_GROUP = "N3151"  
VARIANT = "2-10-8-4"  
  
# Допустимые буквы  
CYRILLIC_LETTERS = ["a", "в", "е", "к", "м", "н", "о", "р", "с", "т", "у", "х"]  
  
def print_version():  
    print(f"{STUDENT_NAME}, гр. {STUDENT_GROUP}")  
    print(f"Вариант: {VARIANT}")  
  
def die(msg):  
    print(msg, file=sys.stderr)  
    sys.exit(1)  
  
def parse_nonnegative_int(s):  
    try:  
        value = int(s, 10)  
    except ValueError:  
        die(f"Ошибка: '{s}' не является числом.")  
  
    if value < 0:  
        die(f"Ошибка: '{s}' не является неотрицательным числом.")
```

```
return value
```

```
def gen_car_number():
```

```
    first = random.choice(CYRILLIC_LETTERS)
```

```
    digits1 = random.randint(100, 999)
```

```
    l1 = random.choice(CYRILLIC_LETTERS)
```

```
    l2 = random.choice(CYRILLIC_LETTERS)
```

```
    region = random.randint(10, 99)
```

```
    return f'{first} {digits1:03d} {l1} {l2} {region:02d}RUS'
```

```
def main():
```

```
    try:
```

```
        opts, args = getopt.getopt(sys.argv[1:], "vn:")
```

```
    except getopt.GetoptError as err:
```

```
        die(f'Ошибка: {err}')
```

```
    num_strings = random.randint(10, 1000)
```

```
    for opt, arg in opts:
```

```
        if opt == "-v":
```

```
            print_version()
```

```
            return
```

```
        if opt == "-n":
```

```
            num_strings = parse_nonnegative_int(arg)
```

```
    if len(args) != 1:
```

```
        die("Ошибка: нужно указать ровно одно имя файла.")
```

```
    filename = args[0]
```

```
    cnt = num_strings
```

```
    # Пустой список => файл нулевого размера
```

```
    if cnt == 0:
```

```

try:
    open(filename, "wb").close()
except OSError as e:
    die(f"Ошибка: не удалось записать файл: {e}.")
print(f"Успешно: файл '{filename}' создан, строк: 0 (файл нулевого размера)")
return

strings = [gen_car_number() for _ in range(cnt)]
encoded = [s.encode("utf-8") + b"\x00" for s in strings]
idxs = random.sample(range(cnt), cnt)

data_first_size = random.randint(4, 32)
data_first = bytes(random.randint(0, 255) for _ in range(data_first_size))
if cnt >= 2:
    data_last_size = random.randint(4, 32)
    data_last = bytes(random.randint(0, 255) for _ in range(data_last_size))
else:
    data_last_size = 0
    data_last = b""

# Смещения о_k строк s_k от начала файла
header_end = 4 + cnt * 6
offs = [0] * cnt
pos = header_end + data_first_size
offs[0] = pos
pos += len(encoded[0])
for k in range(1, cnt):
    if k == cnt - 1 and cnt >= 2:
        pos += data_last_size
    offs[k] = pos
    pos += len(encoded[k])

try:
    with open(filename, "wb") as f:

```

```

f.write(struct.pack("<I", cnt))
for k in range(cnt):
    f.write(struct.pack("<H", idxs[k]))
    f.write(struct.pack("<I", offs[k]))
f.write(data_first)
for k in range(cnt):
    if k == cnt - 1 and cnt >= 2:
        f.write(data_last)
        f.write(encoded[k])
except OSError as e:
    die(f"Ошибка: не удалось записать файл: {e}.")

# Диагностический вывод
print(f"Файл: {filename}")
print(f"cnt = {cnt}")
print(f"Заголовок (i,o)-пар: с 0x{4:x} по 0x{header_end:x} ({cnt * 6} байт)")
print(f"data_first: размер {data_first_size} байт, смещение 0x{header_end:x}")
if cnt >= 2:
    data_last_offset = offs[-1] - data_last_size
    print(f"data_last: размер {data_last_size} байт, смещение 0x{data_last_offset:x}")
else:
    print("data_last: отсутствует (cnt < 2)")
print()
print("Заголовок (k — позиция в файле, idx — позиция в списке, off — смещение строки):")
for k in range(cnt):
    print(f" k={k:4d} idx={idxs[k]:4d} off=0x{offs[k]:08x} ({offs[k]:7d}) s_k={strings[k]}")
print()
print("Порядок строк в списке после загрузки:")
inv = [0] * cnt
for k in range(cnt):
    inv[idxs[k]] = k
for p in range(cnt):
    print(f" list[{p:4d}] = {strings[inv[p]]}")
print(f"\nУспешно: файл '{filename}' создан, строк: {cnt}")

```

```
if __name__ == "__main__":  
    main()
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы была разработана программа на языке С для управления базой данных автомобильных номеров. Реализована структура данных **двусвязный список**, обеспечивающая эффективную вставку и удаление элементов. В соответствии с вариантом реализован алгоритм сортировки **reorder_asc** и механизм работы с бинарными файлами **формата №4**, включающий хранение индексов и смещений для быстрого доступа к данным